

Simple Access Control System

Student number: 2817173

Name: Joanna Chronis

Date: 16 May 2025

Abstract

The aim of this project was to develop an access control system consisting of a keypad, servo motor and 3 LEDs to depict the unlock state of the system, all coded in AVR Assembly. The system needed to compare a user-inputted code against a stored code to determine the unlock state of the system. A basic lock system was successfully created, although it did not meet all the requirements of the project, as it only had a lock and unlock state, and did not give an opportunity to re-enter the code.

1. Introduction

The project requirement was to create an access lock system with 3 possible lock states: Unlocked; Second Attempt; and Locked. By entering the pin code correctly, a servo motor would unlock, and a green LED would illuminate. Entering the pin code incorrectly would put the system into the second-attempt state, where a yellow LED flashed and the user could retry the code. If the code was entered incorrectly again, the system would lock, and the yellow and red LEDs would flash.

The project was required to only make use of timers and interrupts without the use of delays and polling. The project was also required to be coded in pure AVR Assembly language.

This report covers the design of the lock system through both the physical lock system components and the AVR code that controlled said components. The results arising from designing this system are also discussed, explaining obstacle tackled, objectives that were achieved and those that were not and future improvements.

2. Design and methodology

2.1 Overall lock system overview

The overall design of the access lock system is: a 4x4 membrane keypad; 4 red number display LEDs; 4 green counter LEDs; a green LED to display when the code is correct and a red LED to display when the code is incorrect.

A flowchart of the program is shown in Fig 1. The flowchart explains how the program runs.

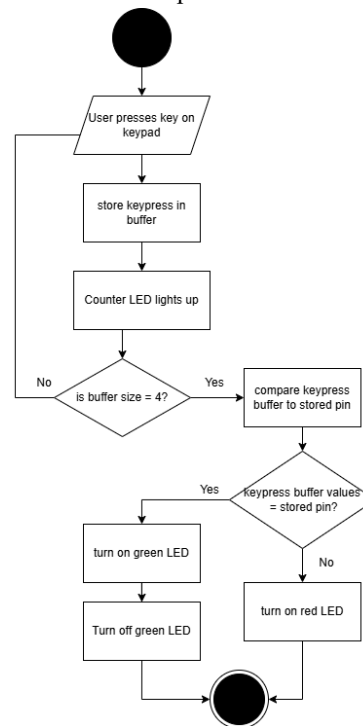


Figure 2: A Flowchart displaying how the program executes

The complete layout of the project components is displayed in Fig. 2

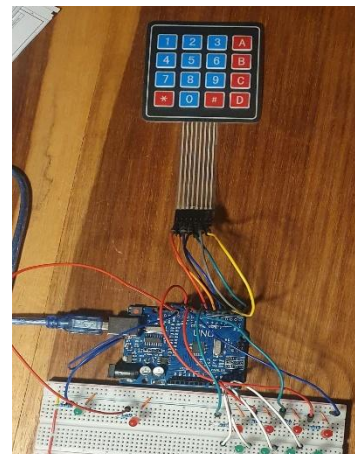


Figure 1: A photograph of the physical system

2.2 Membrane keypad

A 4x4 membrane keypad was used, but was utilized as a 4x3 keypad instead to maximise the number of free pins. This was done by simply removing the scan logic for column 3. The keypad was used with interrupts and debounce verification so as to know when valid key presses were occurring.

Key identification was handled by grounding a single row at a time, showing which column was pulled low too, giving a row-col combination [1]. A lookup table was then

used in order to determine what value each row-col combination pertained to. A sequence was then displayed on the red number display LEDs so the user could know what button they had pressed.

2.3 Counter LEDs

As key values were identified from the keypad, they were stored in a buffer at the current position. [2] A variable tracked which LED within the counter sequence should be on, so that as each key in the pin was pressed, the lighting pattern shifted left and added a new LED as high. When the input was reset, either via a verify or timeout, all the LEDs were turned off.

2.4 Passcode verification method

The correct passcode (1234) was stored as bytes in memory and the user inputted code was stored in a separate buffer (as mentioned previously).

The Z pointer was set to the user input and the Y pointer to the stored passcode, and each digit was compared one by one. If any of these digits did not match, the program jumped to verification failure.

2.5 LED unlock state indicators

Once the user inputted passcode was verified as previously explained, either the green or red LED was triggered. If the code was correct, the green LED illuminated. Likewise, the red LED illuminated if the code was incorrect.

3. Results

Upon running the program with the abovementioned configuration and code techniques, the project successfully collected user input using the membrane keypad and compared the stored code successfully against the inputted one. The green LED lit when the code was correctly inputted, and the red LED lit when the code was incorrectly inputted.

The second-attempt yellow LED and servo motor were not implemented. A delay was also present, all which will be discussed in the following section.

4. Discussion

As mentioned in the Results section, the project was able to successfully determine if the user-entered code was correct, thus making the respective LEDs illuminate. Also mentioned in the Results was how the yellow LED and servo motor were not implemented. Both the yellow LED and servo motor [3] worked perfectly when run separately from the main program, but each time the program code was altered to attempt to include the second-attempt state or servo motor, the verification code would

begin to return incorrectly regardless of the code inputted. This caused the “correct” sequence of the LED and in turn the servo motor to never run, even though the actual logic within the verification code was never changed. In the interest of not losing the validity of the verification code and the functioning of the green LED, it was decided to remove the second-attempt state and servo motor entirely.

The presence of a delay was due to the fact that when the delay was replaced with a timer, the logic of the verification stopped working, similarly to the errors created by the yellow LED and servo motor logic. Due to this, it was a better choice to leave the delay in than remove it, simply so that the rest of the project was able to work.

The issue of the second-attempt, servo motor and delay code bug could have been resolved by starting the project sooner, so as to have ample time to attempt to determine and remedy the problem. Additionally, the inclusion of an would be more user friendly in understanding which key had been pressed, as it could be tedious to try to understand what LED combination pertains to what key was pressed.

5. Conclusion

In conclusion, the project was successful in collecting and comparing a user-inputted code with a stored code. The project lit a green LED if the code was correct and lit a red LED if the code was not. The “second-attempt” yellow LED state and servo motor were not included in the project due to a persistent logic error that was unable to be solved within the span of the project.

Overall, the program functions as an access lock system, since it successfully determines correct code input, regardless of the missing yellow LED state.

References

1. Blogspot.com. (2021). *Assembly via Arduino - Interfacing Keypads*. [online] Available at: <https://akuzechie.blogspot.com/2021/11/assembly-via-arduino-interfacing-keypads.html>. [Accessed 14 May 2025].
2. Avrfreaks.net. (2025). *AVR Freaks*. [online] Available at: <https://www.avrfreaks.net/s/topic/a5C3l000000USA4EAO/t122046> [Accessed 14 May 2025].
3. Blogspot.com. (2021). *Assembly via Arduino - Servo Motor SG90*. [online] Available at: <https://akuzechie.blogspot.com/2021/12/assembly-via-arduino-servo-motor-sg90.html> [Accessed 14 May 2025].